ML 1: Aggregation-Based Models

Victor Veitch

Overview

Examples so far:

- Define parametric model class $\mathscr{F} = \{f_{\theta}\}$ and differentiable loss $L(y, f_{\theta}(\boldsymbol{x}))$
- Optimize with (stochastic) gradient descent

Another approach: combine weak learners into strong learners.

- Approach one: model averaging (e.g., random forests)
- Approach two: iterative construction (e.g., gradient boosting)

Overview

Examples so far:

- Define parametric model class $\mathscr{F} = \{f_{\theta}\}$ and differentiable loss $L(y, f_{\theta}(\boldsymbol{x}))$
- Optimize with (stochastic) gradient descent

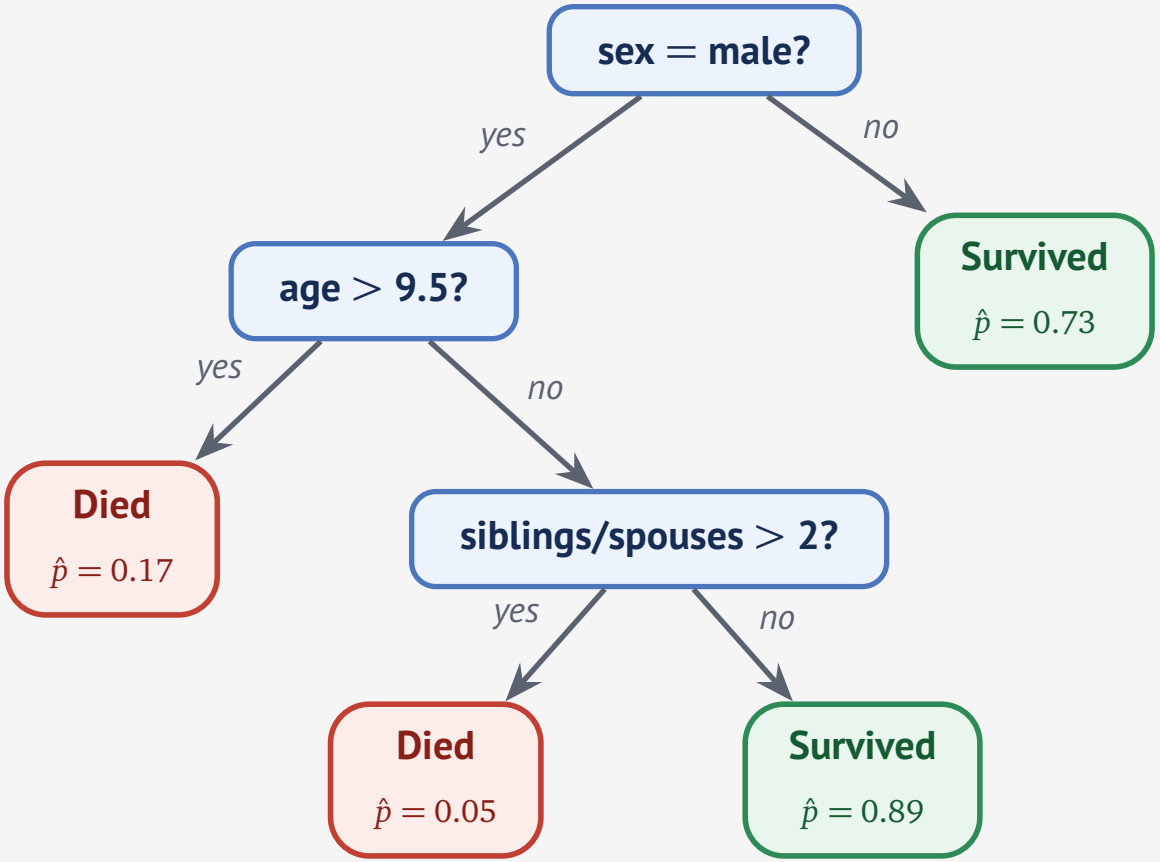
Another approach: combine weak learners into strong learners.

- Approach one: model averaging (e.g., random forests)
- Approach two: iterative construction (e.g., gradient boosting)

Trees

Decision Trees

- Divide feature space into regions—tree structure
- Make fixed prediction in each region—leaf predictions



Titanic passenger survival example (survival rate $\hat{p}$ in each leaf)

Decision Trees

Two parts to a decision tree f:

- Tree structure: partitions feature space into T regions $\text{leaf}_1, \ldots, \text{leaf}_T$
- Leaf values $\{\eta_l\}_{l=1}^{T}$ (prediction in each region)
  - E.g., for regression: $\eta_l = \mathbb{E}[Y \mid \boldsymbol{X} \in \text{leaf}_l]$
  - E.g., for classification: $\eta_l = \text{argmax}_k \, \text{P}(Y = k \mid \boldsymbol{X} \in \text{leaf}_l)$
  - More generally: η_l is parameter of a probabilistic model $p(y \mid \eta_l)$

The function is then:

$$f(\boldsymbol{x}) = \sum_{l=1}^{T} \mathbb{1}[\boldsymbol{x} \in \text{leaf}_l] \, \eta_l$$

Learning Decision Trees

Learn leaf values $\{\eta_l\}_{l=1}^T$ *given tree structure* using $\mathcal{D}^{\text{train}} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$:

- E.g., regression: $\eta_l = \frac{1}{n_l} \sum_{\boldsymbol{x}_i \in \text{leaf}_l} y_i$ (with $n_l = \sum_{i=1}^n \mathbb{1}[\boldsymbol{x}_i \in \text{leaf}_l]$)
- E.g., classification: $\eta_l = \text{argmax}_k \sum_{\boldsymbol{x}_i \in \text{leaf}_l} \mathbb{1}[y_i = k]$
- Generally:

$$\eta_l = \underset{\eta}{\text{argmin}} \sum_{\boldsymbol{x}_i \in \text{leaf}_l} L(y_i, \eta)$$

Leaf values are fixed (and straightforward) given tree structure.

Learning Decision Trees

Learn leaf values $\{\eta_l\}_{l=1}^T$ *given tree structure* using $\mathcal{D}^{\text{train}} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$:

- E.g., regression: $\eta_l = \frac{1}{n_l} \sum_{\boldsymbol{x}_i \in \text{leaf}_l} y_i$ (with $n_l = \sum_{i=1}^n \mathbb{1}[\boldsymbol{x}_i \in \text{leaf}_l]$)
- E.g., classification: $\eta_l = \text{argmax}_k \sum_{\boldsymbol{x}_i \in \text{leaf}_l} \mathbb{1}[y_i = k]$
- Generally:

$$\eta_l = \underset{\eta}{\text{argmin}} \sum_{\boldsymbol{x}_i \in \text{leaf}_l} L(y_i, \eta)$$

Leaf values are fixed (and straightforward) given tree structure.

Learning Decision Trees

How about the tree structure?

- Given tree structure, we know optimal leaf values—so can directly assess empirical risk of a given tree structure.
- Idea: search over all trees for best structure.
- Problem: combinatorial explosion in number of trees!

In practice: use a greedy algorithm to build tree structure.

Learning Decision Trees

How about the tree structure?

- Given tree structure, we know optimal leaf values—so can directly assess empirical risk of a given tree structure.
- Idea: search over all trees for best structure.
- Problem: combinatorial explosion in number of trees!

In practice: use a greedy algorithm to build tree structure.

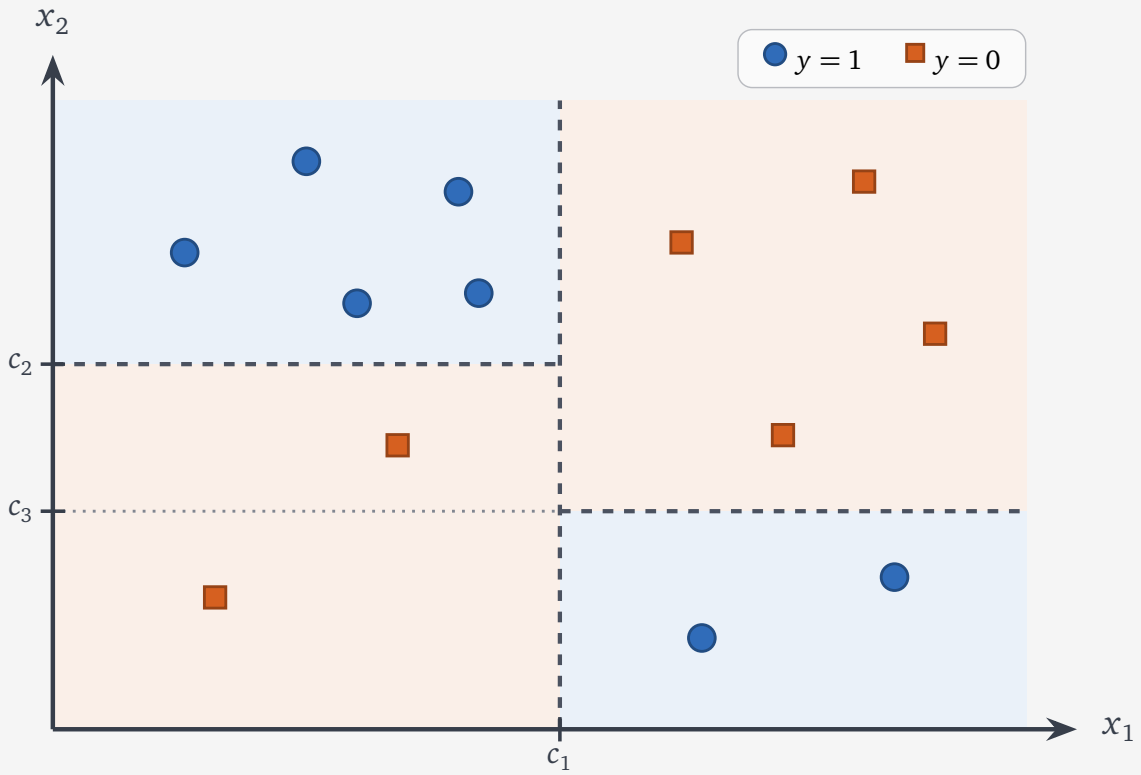
Learning Decision Trees

How about the tree structure?

- Given tree structure, we know optimal leaf values—so can directly assess empirical risk of a given tree structure.
- Idea: search over all trees for best structure.
- Problem: combinatorial explosion in number of trees!

In practice: use a greedy algorithm to build tree structure.

Greedy Tree Building: Basic Idea



1. Find a split rule (feature and threshold) to partition the space
2. Split the data into two subsets according to (1)
3. Recurse separately in each resulting region

Classification Tree

At a given node (with n data points), assume continuous features $\boldsymbol{x} \in \mathbb{R}^p$.

For each feature j:

1. Find a split point c_j^* that minimizes classification error across the two child regions:
   - Region predictions for threshold c:

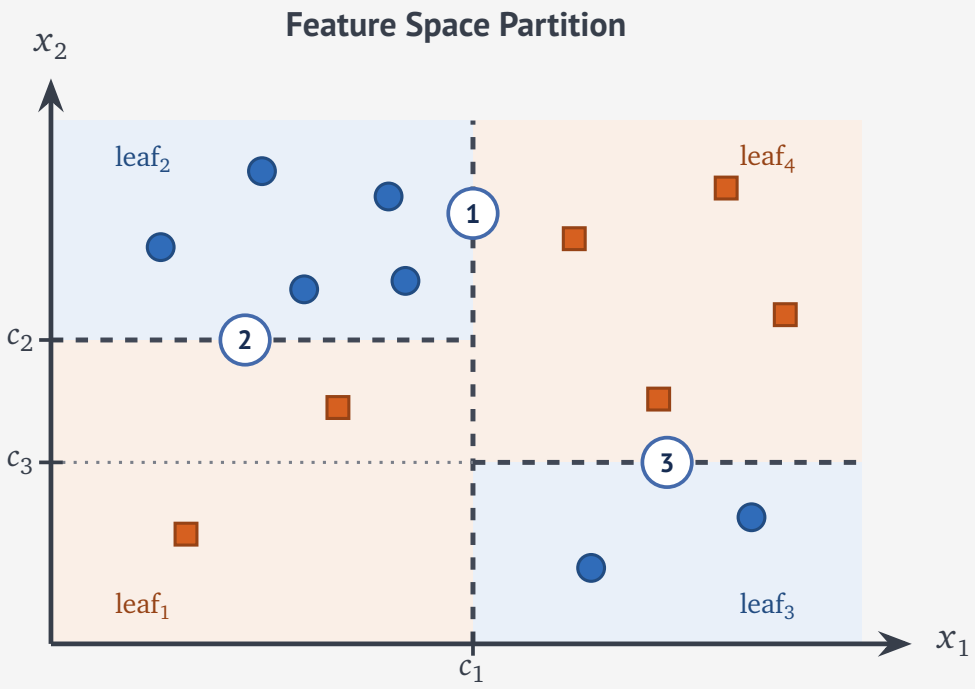
$$\hat{y}_L(j,c) = \text{majority}(\{y_i : x_{i,j} \leq c\}), \quad \hat{y}_R(j,c) = \text{majority}(\{y_i : x_{i,j} > c\})$$

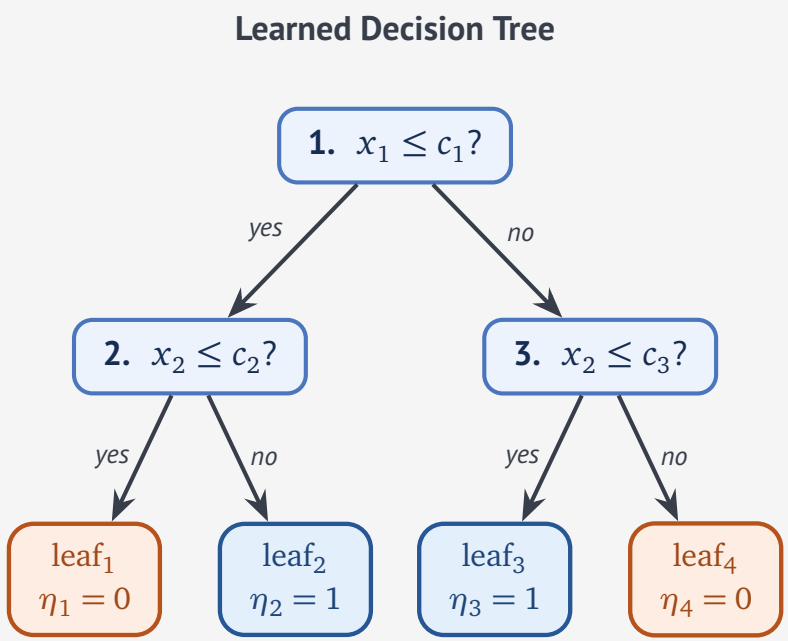
   - Optimal split point for feature j:

$$c_j^* = \underset{c}{\text{argmin}} \frac{1}{n} \sum_{i=1}^{n} \Big(\mathbb{1}[y_i \neq \hat{y}_L(j,c)]\, \mathbb{1}[x_{i,j} \leq c] + \mathbb{1}[y_i \neq \hat{y}_R(j,c)]\, \mathbb{1}[x_{i,j} > c] \Big)$$

2. Choose feature j^* with lowest classification error and split the data at $x_{i,j^*} \leq c_{j^*}^*$

Greedy Tree Building: Example





Regression Trees and Splitting Criteria

How can we use the same ideas to do regression?

Find splits $(j^*, c^*_{j^*})$ that minimize squared error across the two child regions

In general: splitting rule need not exactly match target loss

- E.g., for classification, select split that maximizes "information gain"
- Can yield better final accuracy than greedy splitting on accuracy
- Such heuristics are common in practice

Regression Trees and Splitting Criteria

How can we use the same ideas to do regression?

Find splits $(j^*, c^*_{j^*})$ that minimize squared error across the two child regions

In general: splitting rule need not exactly match target loss

- E.g., for classification, select split that maximizes "information gain"
- Can yield better final accuracy than greedy splitting on accuracy
- Such heuristics are common in practice

Regression Trees and Splitting Criteria

How can we use the same ideas to do regression?

Find splits $(j^*, c^*_{j^*})$ that minimize squared error across the two child regions

In general: splitting rule need not exactly match target loss

- E.g., for classification, select split that maximizes "information gain"
- Can yield better final accuracy than greedy splitting on accuracy
- Such heuristics are common in practice

Overfitting in Decision Trees

What happens if we keep splitting?

- Eventually little data in each leaf region
- Predictions depend on the closest data point

Will this work well?

Overfitting in Decision Trees

What happens if we keep splitting?

- Eventually little data in each leaf region
- Predictions depend on the closest data point

Will this work well?

Controlling Tree Complexity

It will overfit. Fix by

- Requiring multiple data points in each leaf
- Requiring a minimum improvement in loss to split (or pruning)

Is everything okay now?

Controlling Tree Complexity

It will overfit. Fix by

- Requiring multiple data points in each leaf
- Requiring a minimum improvement in loss to split (or pruning)

Is everything okay now?

Instability of Greedy Trees

Greedy algorithms can be unstable

- Later splits heavily influenced by early splits
- If, e.g., two features are similar, which to split on first is somewhat arbitrary
- But can strongly affect final tree

Model Averaging and Random Forests

Combining Weak Learners

- Individual trees unstable / weak predictors
- Idea: combine multiple trees!

Combining Weak Learners

- Individual trees unstable / weak predictors
- Idea: combine multiple trees!

Majority Vote

- Suppose we have B binary predictors $h_1(\boldsymbol{x}), \ldots, h_B(\boldsymbol{x}) \in \{0, 1\}$
- Each has accuracy $p > 0.5$ (better than chance)
- Suppose errors are (somewhat) independent

Majority vote: predict $\hat{y}(\boldsymbol{x}) = \mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$

Majority Vote

- Suppose we have B binary predictors $h_1(\boldsymbol{x}), \ldots, h_B(\boldsymbol{x}) \in \{0, 1\}$
- Each has accuracy $p > 0.5$ (better than chance)
- Suppose errors are (somewhat) independent

Majority vote: predict $\hat{y}(\boldsymbol{x}) = \mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$

Majority Vote

Majority vote: predict $\hat{y}(\boldsymbol{x}) = \mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$

How well does this work?

Chebyshev's inequality: $\mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq t\sqrt{\mathrm{var}(V)}\right) \leq \frac{1}{t^2}$

$\left(\text{i.e., } \mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq \varepsilon\right) \leq \frac{\mathrm{var}(V)}{\varepsilon^2}\right)$

- Let $V = \frac{1}{B}\sum_{b=1}^{B} \mathbb{1}[h_b(\boldsymbol{x}) = y]$ be fraction of correct votes ($\mathbb{E}[V] = p > 0.5$)
- Majority vote errs when $V \leq 0.5 \implies |V - p| \geq p - 0.5$:

$$\mathrm{P}(\text{majority vote errs}) \leq \frac{\mathrm{var}(V)}{(p-0.5)^2}$$

- If errors are independent, $\mathrm{var}(V) = \frac{p(1-p)}{B} \to 0$ as $B \to \infty$

Majority Vote

Majority vote: predict $\hat{y}(\boldsymbol{x}) = \mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$

How well does this work?

Chebyshev's inequality: $\mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq t\sqrt{\mathrm{var}(V)}\right) \leq \frac{1}{t^2}$

$\left(\text{i.e., } \mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq \varepsilon\right) \leq \frac{\mathrm{var}(V)}{\varepsilon^2}\right)$

- Let $V = \frac{1}{B}\sum_{b=1}^{B} \mathbb{1}[h_b(\boldsymbol{x}) = y]$ be fraction of correct votes ($\mathbb{E}[V] = p > 0.5$)
- Majority vote errs when $V \leq 0.5 \implies |V - p| \geq p - 0.5$:

$$\mathrm{P}(\text{majority vote errs}) \leq \frac{\mathrm{var}(V)}{(p-0.5)^2}$$

- If errors are independent, $\mathrm{var}(V) = \frac{p(1-p)}{B} \to 0$ as $B \to \infty$

Majority Vote

Majority vote: predict $\hat{y}(\boldsymbol{x}) = \mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$
How well does this work?

Chebyshev's inequality: $\mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq t\sqrt{\mathrm{var}(V)}\right) \leq \frac{1}{t^2}$
$\left(\text{i.e., } \mathrm{P}\left(\left|V - \mathbb{E}[V]\right| \geq \varepsilon\right) \leq \frac{\mathrm{var}(V)}{\varepsilon^2}\right)$

- Let $V = \frac{1}{B}\sum_{b=1}^{B} \mathbb{1}[h_b(\boldsymbol{x}) = y]$ be fraction of correct votes ($\mathbb{E}[V] = p > 0.5$)
- Majority vote errs when $V \leq 0.5 \implies |V - p| \geq p - 0.5$:

$$\mathrm{P}(\text{majority vote errs}) \leq \frac{\mathrm{var}(V)}{(p-0.5)^2}$$

- If errors are independent, $\mathrm{var}(V) = \frac{p(1-p)}{B} \to 0$ as $B \to \infty$

Random Forests

Want weak predictors that make independent errors.

Two ideas:

- *Bagging* (Bootstrap Aggregation): resample n datapoints with replacement and train on these.
- *Random Feature Subsampling*: at each split (or tree), sample a random subset of candidate features.

Random Forests

Want weak predictors that make independent errors.

Two ideas:

- *Bagging* (Bootstrap Aggregation): resample n datapoints with replacement and train on these.
- *Random Feature Subsampling*: at each split (or tree), sample a random subset of candidate features.

Random Forests

A random forest f is:

1. Create B bootstrap datasets by resampling $\mathscr{D}^{\text{train}}$ with replacement
2. Train a tree h_b on each bootstrap dataset (using greedy splitting with random feature subsets)
3. Aggregate predictions:

$$f(\boldsymbol{x}) = \frac{1}{B} \sum_{b=1}^{B} h_b(\boldsymbol{x})$$

(for regression; or majority vote $\mathbb{1}\left[\frac{1}{B}\sum_{b=1}^{B} h_b(\boldsymbol{x}) > 0.5\right]$ for classification)

Gradient Boosting

Goal

Goal: use additive model $f(\boldsymbol{x}) = \sum_{m=1}^{M} h_m(\boldsymbol{x})$ to solve risk minimization

- Now: want to make "good" choices for each h_m
- Define sequence of partial models $f_m := \sum_{j=1}^{m} h_j = f_{m-1} + h_m$
- Choose h_m to reduce risk of f_{m-1} as much as possible

Contrast with random forests, where each h_b is learned separately in parallel

Goal

Goal: use additive model $f(\boldsymbol{x}) = \sum_{m=1}^{M} h_m(\boldsymbol{x})$ to solve risk minimization

- Now: want to make "good" choices for each h_m
- Define sequence of partial models $f_m := \sum_{j=1}^{m} h_j = f_{m-1} + h_m$
- Choose h_m to reduce risk of f_{m-1} as much as possible

Contrast with random forests, where each h_b is learned separately in parallel

Goal

Goal: use additive model $f(\boldsymbol{x}) = \sum_{m=1}^{M} h_m(\boldsymbol{x})$ to solve risk minimization

- Now: want to make "good" choices for each h_m
- Define sequence of partial models $f_m := \sum_{j=1}^{m} h_j = f_{m-1} + h_m$
- Choose h_m to reduce risk of f_{m-1} as much as possible

Contrast with random forests, where each h_b is learned separately in parallel

Functional Gradients

- Iterative optimization evocative of gradient descent
- Gradient is “best direction” for improvement
- Gradients classically done for vectors
- Functions are infinite vectors

Can we do gradient optimization on functions?

Functional Gradients

$$\mathscr{L}(f) = \mathbb{E}_{\boldsymbol{X},Y}\Big[L(Y,f(\boldsymbol{X}))\Big] = \mathbb{E}_{\boldsymbol{X}}\Big[\mathbb{E}_{Y|\boldsymbol{X}}\Big[L(Y,f(\boldsymbol{X}))\Big]\Big]$$

Suppose we have a current f_{m-1}. Derivative at this function is

$$g_m(\boldsymbol{x}) = \frac{\partial \mathbb{E}[L(Y,f(\boldsymbol{x}))\,|\,\boldsymbol{x}]}{\partial f(\boldsymbol{x})}\Big|_{f=f_{m-1}}$$

Interchanging differentiation and integration:

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y,f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})}\,|\,\boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

Functional Gradients

Gradient is

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

Example: $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$

$$\begin{aligned} g_m(\boldsymbol{x}) &= \mathbb{E}\left[\frac{\partial\left[\frac{1}{2}(Y - f(\boldsymbol{x}))^2\right]}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right] \end{aligned}$$

Negative gradient $-g_m(\boldsymbol{x}) = \mathbb{E}[Y - f_{m-1}(\boldsymbol{x}) \,|\, \boldsymbol{x}]$ is expected residual of f_{m-1}

Functional Gradients

Gradient is

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

Example: $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$

$$\begin{aligned} g_m(\boldsymbol{x}) &= \mathbb{E}\left[\frac{\partial\left[\frac{1}{2}(Y - f(\boldsymbol{x}))^2\right]}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right] \end{aligned}$$

Negative gradient $-g_m(\boldsymbol{x}) = \mathbb{E}[Y - f_{m-1}(\boldsymbol{x}) \,|\, \boldsymbol{x}]$ is expected residual of f_{m-1}

Functional Gradients

Gradient is

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

Example: $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$

$$\begin{aligned} g_m(\boldsymbol{x}) &= \mathbb{E}\left[\frac{\partial\left[\frac{1}{2}(Y - f(\boldsymbol{x}))^2\right]}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}} \\ &= \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right] \end{aligned}$$

Negative gradient $-g_m(\boldsymbol{x}) = \mathbb{E}[Y - f_{m-1}(\boldsymbol{x}) \,|\, \boldsymbol{x}]$ is expected residual of f_{m-1}

Functional Gradients

Get mth partial model by stepping in the negative gradient direction:

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) - \rho_m g_m(\boldsymbol{x})$$

$\rho_m > 0$ is *step size*.

Can be set by line search to minimize loss

With $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$,

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \mid \boldsymbol{x}\right]$$

Problems?

Functional Gradients

Get mth partial model by stepping in the negative gradient direction:

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) - \rho_m g_m(\boldsymbol{x})$$

$\rho_m > 0$ is *step size*.

Can be set by line search to minimize loss

With $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$,

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]$$

Problems?

Functional Gradients

Get mth partial model by stepping in the negative gradient direction:

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) - \rho_m g_m(\boldsymbol{x})$$

$\rho_m > 0$ is *step size*.

Can be set by line search to minimize loss

With $L(y, f(\boldsymbol{x})) = \frac{1}{2}(y - f(\boldsymbol{x}))^2$,

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]$$

Problems?

Functional Gradients

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[f_{m-1}(\boldsymbol{x}) - Y \,|\, \boldsymbol{x}\right]$$

Need to learn this!

- $g_m(\boldsymbol{x})$ depends on true data-generating distribution
- Idea: estimate using training data
- Basically run model fit subroutine at each step

Greedy Function Optimization

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \mid \boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

To estimate g_m:

1. Define pointwise derivative at training points:

$$\gamma(y_i, \boldsymbol{x}_i) := \frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\Big|_{f=f_{m-1}}$$

2. Observe that

$$g_m(\boldsymbol{x}) = \mathbb{E}[\gamma(Y, \boldsymbol{x}) \mid \boldsymbol{x}]$$

3. So estimate over base learner class $\mathcal{H}$ as

$$\hat{g}_m = \underset{h \in \mathcal{H}}{\operatorname{argmin}} \sum_{i=1}^{n} \left(\gamma(y_i, \boldsymbol{x}_i) - h(\boldsymbol{x}_i)\right)^2$$

Greedy Function Optimization

$$g_m(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial L(Y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \,|\, \boldsymbol{x}\right]\Big|_{f=f_{m-1}}$$

To estimate g_m:

1. Define pointwise derivative at training points:

$$\gamma(y_i, \boldsymbol{x}_i) := \frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\Big|_{f=f_{m-1}}$$

2. Observe that

$$g_m(\boldsymbol{x}) = \mathbb{E}[\gamma(Y, \boldsymbol{x}) \mid \boldsymbol{x}]$$

3. So estimate over base learner class $\mathscr{H}$ as

$$\hat{g}_m = \operatorname*{argmin}_{h \in \mathscr{H}} \sum_{i=1}^{n} \big(\gamma(y_i, \boldsymbol{x}_i) - h(\boldsymbol{x}_i)\big)^2$$

Greedy Function Optimization

Update f_{m-1} by setting $f_m = f_{m-1} - \rho_m \hat{g}_m$ (i.e., $h_m = -\rho_m \hat{g}_m$), where

$$\rho_m = \underset{\rho}{\operatorname{argmin}} \sum_{i=1}^{n} L\big(y_i, f_{m-1}(\boldsymbol{x}_i) - \rho \hat{g}_m(\boldsymbol{x}_i)\big)$$

- A kind of projected functional gradient descent
- Step size set by line search (or fixed learning rate $\rho > 0$)

Example: Gradient Boosted Regression Trees

Loss: Squared error

$$L(y_i, f(\boldsymbol{x}_i)) = \frac{1}{2}(y_i - f(\boldsymbol{x}_i))^2$$

Current predictor:

$$f_{m-1}(\boldsymbol{x}) = \sum_{j=1}^{m-1} h_j(\boldsymbol{x})$$

Compute derivative at f_{m-1}:

$$\gamma_i := \left.\frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right|_{f=f_{m-1}} = f_{m-1}(\boldsymbol{x}_i) - y_i$$

Need to project derivatives onto trees

Example: Gradient Boosted Regression Trees

How do we project derivatives with trees?

$$\gamma_i := f_{m-1}(\boldsymbol{x}_i) - y_i$$

- Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^n$ (minimizing MSE)
- Update $f_m = f_{m-1} - \rho_m \hat{g}_m$

(Equivalently, fit a tree directly to residuals $r_i = y_i - f_{m-1}(\boldsymbol{x}_i)$ and add $\rho_m \hat{r}_m$.)

Example: Gradient Boosted Regression Trees

How do we project derivatives with trees?

$$\gamma_i := f_{m-1}(\boldsymbol{x}_i) - y_i$$

- Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^n$ (minimizing MSE)
- Update $f_m = f_{m-1} - \rho_m \hat{g}_m$

(Equivalently, fit a tree directly to residuals $r_i = y_i - f_{m-1}(\boldsymbol{x}_i)$ and add $\rho_m \hat{r}_m$.)

General Gradient Boosting

This strategy generalizes to other losses (absolute loss, negative log-likelihoods, etc.):

1. Define pseudo-outcomes γ_i:

$$\gamma_i := \frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\Big|_{f=f_{m-1}}$$

2. Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^n$ (using squared error objective)

3. Set $f_m = f_{m-1} - \rho_m \hat{g}_m$
   (with ρ_m chosen by line search, or fixed step size $\rho > 0$)

Gradient Boosting for Classification

How about binary classification ($y_i \in \{0, 1\}$)?

- Predicted probability: $\tilde{f}_m(\boldsymbol{x}) = \sigma(f_m(\boldsymbol{x})) = \sigma\left(\sum_{j=1}^{m} h_j(\boldsymbol{x})\right)$
- Loss: $L(y, f(\boldsymbol{x})) = -y \log \sigma(f(\boldsymbol{x})) - (1-y) \log\left(1 - \sigma(f(\boldsymbol{x}))\right)$
- Derivative at f_{m-1}:

$$\begin{aligned}\left.\frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right|_{f=f_{m-1}} &= \frac{-y_i}{1 + \exp(f_{m-1}(\boldsymbol{x}_i))} + \frac{1-y_i}{1 + \exp(-f_{m-1}(\boldsymbol{x}_i))} \\ &= \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i\end{aligned}$$

1. Compute pseudo-outcomes $\gamma_i := \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i$
2. Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^{n}$ (using squared error objective)
3. Set $f_m = f_{m-1} - \rho_m \hat{g}_m$

Gradient Boosting for Classification

How about binary classification ($y_i \in \{0, 1\}$)?

- Predicted probability: $\tilde{f}_m(\boldsymbol{x}) = \sigma(f_m(\boldsymbol{x})) = \sigma\left(\sum_{j=1}^{m} h_j(\boldsymbol{x})\right)$
- Loss: $L(y, f(\boldsymbol{x})) = -y \log \sigma(f(\boldsymbol{x})) - (1-y) \log\left(1 - \sigma(f(\boldsymbol{x}))\right)$
- Derivative at f_{m-1}:

$$\begin{aligned}\left.\frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right|_{f=f_{m-1}} &= \frac{-y_i}{1+\exp(f_{m-1}(\boldsymbol{x}_i))} + \frac{1-y_i}{1+\exp(-f_{m-1}(\boldsymbol{x}_i))} \\ &= \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i\end{aligned}$$

1. Compute pseudo-outcomes $\gamma_i := \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i$
2. Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^{n}$ (using squared error objective)
3. Set $f_m = f_{m-1} - \rho_m \hat{g}_m$

Gradient Boosting for Classification

How about binary classification ($y_i \in \{0, 1\}$)?

- Predicted probability: $\tilde{f}_m(\boldsymbol{x}) = \sigma(f_m(\boldsymbol{x})) = \sigma\left(\sum_{j=1}^{m} h_j(\boldsymbol{x})\right)$
- Loss: $L(y, f(\boldsymbol{x})) = -y \log \sigma(f(\boldsymbol{x})) - (1-y) \log\left(1 - \sigma(f(\boldsymbol{x}))\right)$
- Derivative at f_{m-1}:

$$\begin{aligned}\left.\frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\right|_{f=f_{m-1}} &= \frac{-y_i}{1+\exp(f_{m-1}(\boldsymbol{x}_i))} + \frac{1-y_i}{1+\exp(-f_{m-1}(\boldsymbol{x}_i))} \\ &= \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i\end{aligned}$$

1. Compute pseudo-outcomes $\gamma_i := \sigma(f_{m-1}(\boldsymbol{x}_i)) - y_i$
2. Fit a regression tree $\hat{g}_m$ on $\{(\boldsymbol{x}_i, \gamma_i)\}_{i=1}^{n}$ (using squared error objective)
3. Set $f_m = f_{m-1} - \rho_m \hat{g}_m$

Questions

- Why use trees?
- Should we do data subsampling for tree fitting? Feature subsampling?
- For f_m learned by minimizing logistic risk, does $f_m(\boldsymbol{x}) \approx \mathrm{P}(Y = 1 \mid \boldsymbol{x})$?

XGBoost

eXtreme Gradient Boosting (XGBoost)

- Particular variant of boosted trees
- Workhorse of ML with tabular data
  E.g., main ingredient for winning Kaggle solutions
- Uses clever tweak to gradient boosting above
  (2nd-order boosting existed pre-XGBoost;
  main contribution was implementation + clever computational ideas)

Boosting Redux

- Goal at step m: minimize $\hat{\mathscr{L}}^m(h) = \frac{1}{n}\sum_{i=1}^n L\big(y_i, f_{m-1}(\boldsymbol{x}_i) + h(\boldsymbol{x}_i)\big)$
- Idea: second-order Taylor approximation

$$\hat{\mathscr{L}}^m(h) \approx \frac{1}{n}\sum_{i=1}^n \left(g_i h(\boldsymbol{x}_i) + \frac{1}{2} b_i h(\boldsymbol{x}_i)^2 \right) + \frac{1}{n}\sum_{i=1}^n L\big(y_i, f_{m-1}(\boldsymbol{x}_i)\big)$$

  where

$$g_i = \frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\bigg|_{f=f_{m-1}}, \qquad b_i = \frac{\partial^2 L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)^2}\bigg|_{f=f_{m-1}}$$

- Pick h_m to minimize the second-order approximation, and set $f_m = f_{m-1} + h_m$ (or $f_{m-1} + \rho h_m$)

Boosting Redux

- Goal at step m: minimize $\hat{\mathscr{L}}^m(h) = \frac{1}{n}\sum_{i=1}^n L\big(y_i, f_{m-1}(\boldsymbol{x}_i) + h(\boldsymbol{x}_i)\big)$
- Idea: second-order Taylor approximation

$$\hat{\mathscr{L}}^m(h) \approx \frac{1}{n}\sum_{i=1}^n \left(g_i h(\boldsymbol{x}_i) + \frac{1}{2} b_i h(\boldsymbol{x}_i)^2 \right) + \frac{1}{n}\sum_{i=1}^n L\big(y_i, f_{m-1}(\boldsymbol{x}_i)\big)$$

  where

$$g_i = \frac{\partial L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)}\bigg|_{f=f_{m-1}}, \qquad b_i = \frac{\partial^2 L(y_i, f(\boldsymbol{x}_i))}{\partial f(\boldsymbol{x}_i)^2}\bigg|_{f=f_{m-1}}$$

- Pick h_m to minimize the second-order approximation, and set $f_m = f_{m-1} + h_m$ (or $f_{m-1} + \rho h_m$)

Why Trees?

Notation:

- Tree with T leaves and leaf weights $\{\eta_l\}_{l=1}^T$
- Predictions are $h(\boldsymbol{x}) = \sum_{l=1}^T \mathbb{1}[\boldsymbol{x} \in \text{leaf}_l]\, \eta_l$
- Training indices in leaf l: $I_l = \{i : \boldsymbol{x}_i \in \text{leaf}_l\}$

Drop constants independent of h and rewrite objective:

$$\tilde{\mathscr{L}}^m(h) := \sum_{i=1}^n \left(g_i h(\boldsymbol{x}_i) + \frac{1}{2} b_i h(\boldsymbol{x}_i)^2 \right)$$

$$= \sum_{l=1}^T \left[\left(\sum_{i \in I_l} g_i \right) \eta_l + \frac{1}{2} \left(\sum_{i \in I_l} b_i \right) \eta_l^2 \right]$$

Why Trees?

Notation:

- Tree with T leaves and leaf weights $\{\eta_l\}_{l=1}^T$
- Predictions are $h(\boldsymbol{x}) = \sum_{l=1}^T \mathbb{1}[\boldsymbol{x} \in \text{leaf}_l]\, \eta_l$
- Training indices in leaf l: $I_l = \{i : \boldsymbol{x}_i \in \text{leaf}_l\}$

Drop constants independent of h and rewrite objective:

$$\begin{aligned}\tilde{\mathscr{L}}^m(h) &:= \sum_{i=1}^n \left(g_i h(\boldsymbol{x}_i) + \frac{1}{2} b_i h(\boldsymbol{x}_i)^2 \right) \\ &= \sum_{l=1}^T \left[\left(\sum_{i \in I_l} g_i \right) \eta_l + \frac{1}{2} \left(\sum_{i \in I_l} b_i \right) \eta_l^2 \right]\end{aligned}$$

Why Trees?

To find best tree, rewrite objective:

$$\tilde{\mathscr{L}}^m(h) = \sum_{l=1}^{T}\Big[\Big(\sum_{i\in I_l} g_i\Big)\eta_l + \frac{1}{2}\Big(\sum_{i\in I_l} b_i\Big)\eta_l^2\Big]$$

For fixed tree structure, the optimal leaf weight η_l^* is:

$$\eta_l^* = -\frac{\sum_{i\in I_l} g_i}{\sum_{i\in I_l} b_i}$$

So only free parameter is tree structure. And optimal objective value for a given tree is:

$$\tilde{\mathscr{L}}^m(\text{tree}) = -\frac{1}{2}\sum_{l=1}^{T}\frac{\big(\sum_{i\in I_l} g_i\big)^2}{\sum_{i\in I_l} b_i}$$

Gives cheap rule for greedy splitting procedure!
That is, trees happen to be easy to fit in 2nd-order boosting.

Why Trees?

To find best tree, rewrite objective:

$$\tilde{\mathscr{L}}^m(h) = \sum_{l=1}^{T}\left[\left(\sum_{i\in I_l} g_i\right)\eta_l + \frac{1}{2}\left(\sum_{i\in I_l} b_i\right)\eta_l^2\right]$$

For fixed tree structure, the optimal leaf weight η_l^* is:

$$\eta_l^* = -\frac{\sum_{i\in I_l} g_i}{\sum_{i\in I_l} b_i}$$

So only free parameter is tree structure. And optimal objective value for a given tree is:

$$\tilde{\mathscr{L}}^m(\text{tree}) = -\frac{1}{2}\sum_{l=1}^{T}\frac{\left(\sum_{i\in I_l} g_i\right)^2}{\sum_{i\in I_l} b_i}$$

Gives cheap rule for greedy splitting procedure!
That is, trees happen to be easy to fit in 2nd-order boosting.

Why Trees?

To find best tree, rewrite objective:

$$\tilde{\mathscr{L}}^m(h) = \sum_{l=1}^{T}\Big[\Big(\sum_{i\in I_l} g_i\Big)\eta_l + \frac{1}{2}\Big(\sum_{i\in I_l} b_i\Big)\eta_l^2\Big]$$

For fixed tree structure, the optimal leaf weight η_l^* is:

$$\eta_l^* = -\frac{\sum_{i\in I_l} g_i}{\sum_{i\in I_l} b_i}$$

So only free parameter is tree structure. And optimal objective value for a given tree is:

$$\tilde{\mathscr{L}}^m(\text{tree}) = -\frac{1}{2}\sum_{l=1}^{T}\frac{\big(\sum_{i\in I_l} g_i\big)^2}{\sum_{i\in I_l} b_i}$$

Gives cheap rule for greedy splitting procedure!
That is, trees happen to be easy to fit in 2nd-order boosting.

Why Trees?

To find best tree, rewrite objective:

$$\tilde{\mathscr{L}}^m(h) = \sum_{l=1}^{T}\Big[\Big(\sum_{i\in I_l} g_i\Big)\eta_l + \frac{1}{2}\Big(\sum_{i\in I_l} b_i\Big)\eta_l^2\Big]$$

For fixed tree structure, the optimal leaf weight η_l^* is:

$$\eta_l^* = -\frac{\sum_{i\in I_l} g_i}{\sum_{i\in I_l} b_i}$$

So only free parameter is tree structure. And optimal objective value for a given tree is:

$$\tilde{\mathscr{L}}^m(\text{tree}) = -\frac{1}{2}\sum_{l=1}^{T}\frac{\big(\sum_{i\in I_l} g_i\big)^2}{\sum_{i\in I_l} b_i}$$

Gives cheap rule for greedy splitting procedure!
That is, trees happen to be easy to fit in 2nd-order boosting.